



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 中断

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/3/23-2026/3/28

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
五、实验总结与心得体会.....	10
六、附录：参考资料清单.....	10

Section 1 实验概述

学习一份 C 代码是如何通过预编译、编译、汇编和链接生成最终的可执行文件、学习 C/C++ 项目管理方案、C 和汇编混合编程方法（如何在 C 代码中调用汇编代码编写的函数和如何在汇编代码中调用使用 C 编写的函数）。然后进入本章主要内容——中断，首先了解保护模式下的中断处理机制和可编程中断部件 8259A 芯片。最后，通过编写实时钟中断处理函数来将本章的所有内容串联起来。

通过本章的学习，同学们将掌握使用 C 语言来编写内核的方法，理解保护模式的中断处理机制和处理时钟中断，为后面的二级分页机制和多线程/进程打下基础。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++/C 程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）/ Gedit（图形化）
 - 代码编译工具：GCC、g++、nasm、ld（链接器）、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：复现“一个混合编程的例子”，C/C++/汇编代码的混合编程
- 实验任务 2：复现“内核的加载”，使用 C/C++ 编写内核
- 实验任务 3：复现“初始化 IDT”，实现 IDT 初始化以及中断处理
- 实验任务 4：复现“8259A 编程”，实现时钟中断处理过程

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：混合编程的基本思路
1. 复现“一个混合编程的例子”，并修改最终输出
 2. 结合具体的代码说明 C 代码调用汇编函数的语法和汇编代码调用 C 函数的语法。例如，结合关键代码说明 `global`、`extern` 关键字的作用，为什么 C++ 的

函数前需要加上 `extern "C"` 等

3. 学习 make 的使用，并用make 来构建项目

● 思路分析：

1. 避免函数重定义问题：在.h 文件中编写函数的声明，然后在.c 或.cpp 文件中实现这个函数

2. 编译工具——make。实际上， make 并不提供编译工具，依旧使用 gcc/g++来编译， make 只是把这些编译命令合在了一个文件中，称为 Makefile，**最终目标（main.out）放最顶部**

3. 当我们需要在汇编代码中使用 C/C++函数时，我们只需要在汇编代码中声明这个函数来自于外部。我们并不需要关心函数是否被实现，因为我们在链接阶段才会用到函数的实现。同理，当我们需要在 C/C++使用汇编代码的函数时，我们也只需要在 C/C++中声明用到的函数来自于外部即可

4.

- 如果函数有参数，那么参数从右向左依次入栈。
- 如果函数有返回值，返回值放在 eax 中。
- 放置于栈的参数一般使用 ebp 来获取

当我们需要在汇编代码中调用函数 `function_from_C`，调用的形式是 `function_from_C(1,2)` 此时的汇编代码如下。

```
push 2      ; arg2
push 1      ; arg1
call function_from_C
add esp, 8   ; 清除栈上的参数
```

● 实验步骤：

①在文件 `c_func.c` 中定义 C 函数 `function_from_C`

```
#include <stdio.h>

void function_from_C() {
    printf("This is a function from C.\n");
}
```

②在文件 `cpp_func.cpp` 中定义 C++函数 `function_from_CPP`

▲`extern "C"`目的是告诉编译器按 C 代码的规则编译，不进行名字修饰，保证编译后仍为原来的函数名

```
#include <iostream>

extern "C" void function_from_CPP() {
    std::cout << "This is a function from C++." << std::endl;
}
```

③在文件 asm_func.asm 中定义汇编函数 function_from_asm, 在 function_from_asm 中调用 function_from_C 和 function_from_CPP

▲global: 使得在 C 与汇编链接阶段能够找到函数的实现

```
[bits 32]
global function_from_asm
extern function_from_C
extern function_from_CPP

function_from_asm:
    call function_from_C
    call function_from_CPP
    ret
```

④在文件 main.cpp 中调用汇编函数 function_from_asm

▲extern: 声明函数来自外部

```
#include <iostream>

extern "C" void function_from_asm();

int main() {
    std::cout << "Call function from assembly." << std::endl;
    文件 function_from_asm();
    std::cout << "Done by 24325242 txc." << std::endl;
}
```

⑤编写 Makefile 文件

```
main.out: main.o c_func.o cpp_func.o asm_func.o
    g++ -o main.out main.o c_func.o cpp_func.o asm_func.o -m32

c_func.o: c_func.c
    gcc -o c_func.o -m32 -c c_func.c

cpp_func.o: cpp_func.cpp
    g++ -o cpp_func.o -m32 -c cpp_func.cpp

main.o: main.cpp
    g++ -o main.o -m32 -c main.cpp

asm_func.o: asm_func.asm
    nasm -o asm_func.o -f elf32 asm_func.asm

clean:
    rm *.o *.out
```

⑥输入指令 make 然后运行

● 实验结果展示:

```

lalala@lalala-VirtualBox:~/lab4/src/2$ make
g++ -o main.o -m32 -c main.cpp
gcc -o c_func.o -m32 -c c_func.c
g++ -o cpp_func.o -m32 -c cpp_func.cpp
nasm -o asm_func.o -f elf32 asm_func.asm
g++ -o main.out main.o c_func.o cpp_func.o asm_func.o -m32
lalala@lalala-VirtualBox:~/lab4/src/2$ ./main.out
Call function from assembly.
This is a function from C.
This is a function from C++.
Done by 24325242 txc.

```

----- 实验任务 2 -----

- 任务要求：使用 C/C++ 编写内核——在 bootloader 中加载操作系统内核到地址 0x20000，然后跳转到 0x20000。内核接管控制权后，输出“学号+姓名首字母”。

- 思路分析：

1. C/C++ 项目管理

在 project 文件夹下会有 build, include, run, src 等，各个子文件夹的含义如下。

- project/build。存放 Makefile, make 之后生成的中间文件如.o, .bin 等会放置在这里，目的是防止这些文件混在代码文件中。
- project/include。存放.h 等函数定义和常量定义的头文件等。
- project/run。存放 gdb 配置文件，硬盘映像.img 文件等。
- project/src。存放.c, .cpp 等函数实现的文件。

2. 注意需要在 build 文件夹编译并且运行

3. 注意汇编代码需要 global，而 C++ 需要 extern “C”

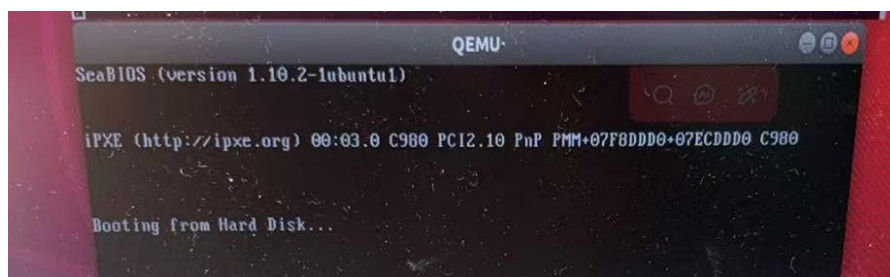
4. 在本次实验中最开始出现无法输出的问题，最开始是因为以下原因

```

WARNING: Image format was not specified for '../run/hd.img' and probing guessed
raw.
Automatically detecting the format is dangerous for raw images, write o
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.

```

后来一直不能输出是因为一直在旧的硬盘镜像（坏的）进行代码写入



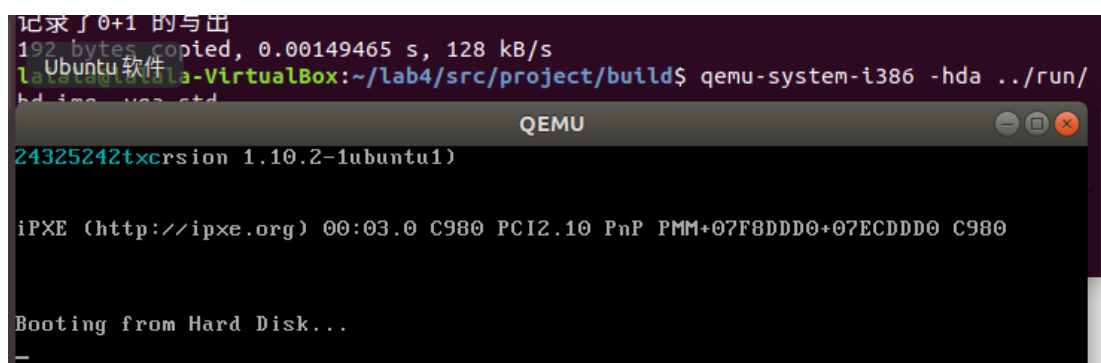
需要使用指令

```
cd project1/run
rm hd.img
dd if=/dev/zero of=hd.img bs=1M count=10
cd ../build
make clean
make build
qemu-system-i386 -hda ../run/hd.img -vga std
```

● 实验步骤：

- ①新建代码管理结构目录
- ②编写代码（详见附件）
- ③编写 makefile 文件
- ④手动创建 **hd.img**
- ⑤在 build 文件夹 make && make run 进行运行得到结果

● 实验结果展示：



----- 实验任务 3 -----

- 任务要求：复现网址中“初始化 IDT”部分，你可以更改默认的中断处理函数为你编写的函数（调用处理函数时输出个人学号或姓名信息），然后触发
- 思路分析：

1. 注意在 asm_utils.h 文件里面声明 extern

```
lalala@lalala-VirtualBox:~/lab4/src/project/build$ make
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I.
./include -c ./src/kernel/setup.cpp ./src/kernel/interrupt.cpp
./src/kernel/setup.cpp: In function 'void setup_kernel()':
./src/kernel/setup.cpp:16:5: error: 'asm_halt' was not declared in this scope
    asm_halt();
    ^~~~~~
./src/kernel/interrupt.cpp: In member function 'void InterruptManager::initialize()':
./src/kernel/interrupt.cpp:15:5: error: 'asm_lidt' was not declared in this scope
    asm_lidt(IDT_START_ADDRESS, 256 * 8 - 1);
    ^~~~~~
./src/kernel/interrupt.cpp:19:43: error: 'asm_unhandled_interrupt' was not declared in this scope
    setInterruptDescriptor(i, (uint32)asm_unhandled_interrupt, 0);
                                   ^~~~~~~~~~~~~~~~~~~~~~
```

2. lidt 实际上是将以 tag 为起始地址的 48 字节放入到寄存器 IDTR 中。由于我们打算在 C 代码中初始化 IDT，而 C 语言的语法并未提供 lidt 语句。因此我们需要在汇编代码中实现能够将 IDT 的信息放入到 IDTR 的函数 asm_lidt

● 实验步骤：

①编写 include/interrupt.h 文件

②编写 interrupt.cpp 代码

③定义默认终端处理函数

④在函数 src/kernel/setup_kernel.cpp 中定义并初始化中断处理器

⑤编写 os_modules.h 和 os_constant.h 文件(注意 IDT_START_ADDRESS, 初始地址的设定)

```
#ifndef OS_CONSTANT_H
#define OS_CONSTANT_H

#define IDT_START_ADDRESS 0x8880
#define CODE_SELECTOR 0x20

#endif
```

⑥使用 debug 来查看我们是否已经放入默认的中断描述符

⑦修改 setup.cpp 看中断是否能被触发

```

#include "asm_utils.h"
#include "interrupt.h"

// 中断管理器
InterruptManager interruptManager;

extern "C" void setup_kernel()
{
    // 中断处理部件
    interruptManager.initialize();

    // 尝试触发除0错误
    int a = 1 / 0;

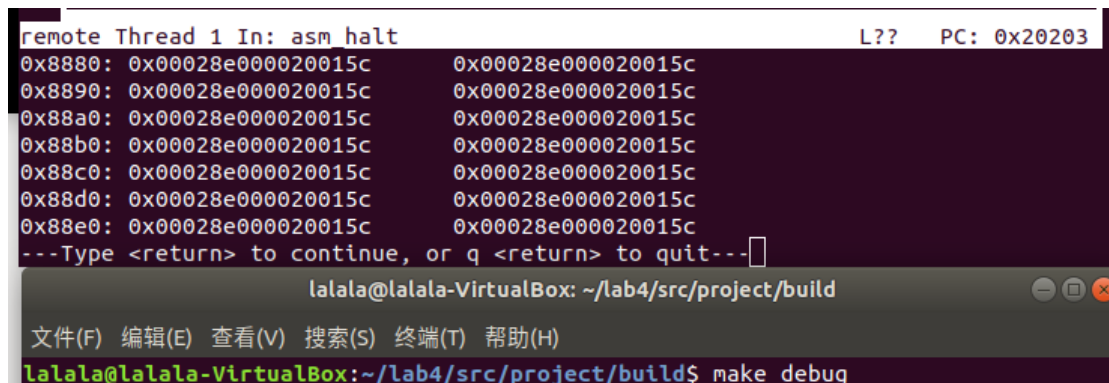
    // 死循环
    asm_halt();
}

```

- 实验结果展示：通过执行前述代码，可得下图结果。

①中断描述符

▲此处结尾与示例不同，但是默认中断描述符为固定的 0x8E00 高 16 位，最后 6 位是中断处理函数地址（因编译不同而不一样，与代码布局、函数位置、链接脚本相关……）

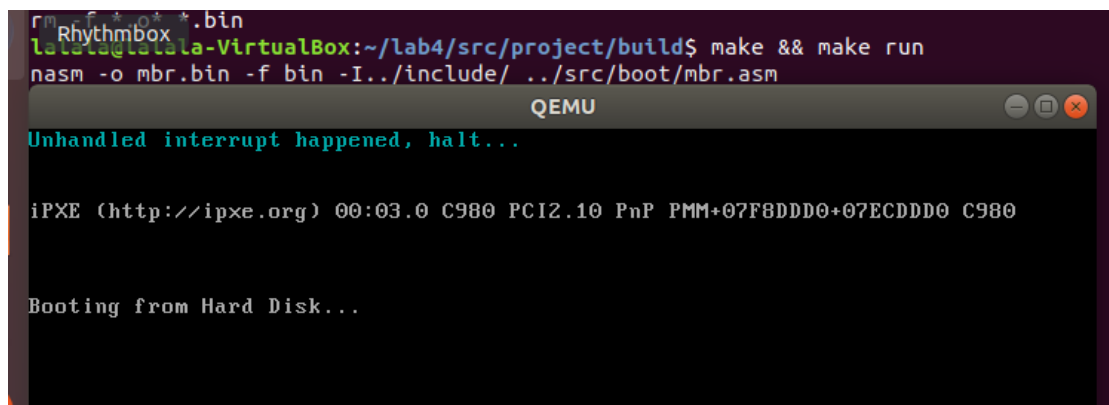


```

remote Thread 1 In: asm halt
0x8880: 0x00028e000020015c    0x00028e000020015c
0x8890: 0x00028e000020015c    0x00028e000020015c
0x88a0: 0x00028e000020015c    0x00028e000020015c
0x88b0: 0x00028e000020015c    0x00028e000020015c
0x88c0: 0x00028e000020015c    0x00028e000020015c
0x88d0: 0x00028e000020015c    0x00028e000020015c
0x88e0: 0x00028e000020015c    0x00028e000020015c
---Type <return> to continue, or q <return> to quit---
lalala@lalala-VirtualBox: ~/lab4/src/project/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
lalala@lalala-VirtualBox:~/lab4/src/project/build$ make debug

```

②触发除 0 异常证明 asm_unhandled_interrupt 可以正常工作



```

rm -f *.bin
lalala@lalala-VirtualBox:~/lab4/src/project/build$ make && make run
nasm -o mbr.bin -f bin -I../include/ ../src/boot/mbr.asm

QEMU
Unhandled interrupt happened, halt...

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDD0+07ECDDD0 C980

Booting from Hard Disk...

```

----- 实验任务 4 -----

- 任务要求：时钟中断的处理。复现“8259A 编程—实时钟中断的处理”部分，

要求：仿照该章节中使用C 语言来实现时钟中断的例子，利用 C/C++ 、 InterruptManager 、 STDIO 和你自己封装的类来实现你的时钟中断处理过程(例如，通过时钟中断，你可以在屏幕的第一行实现一个跑马灯。跑马灯显示自己学号和英文名，即类似于 LED 屏幕显示的效果)

注意：不要使用纯汇编的方式来实现

● 实验步骤：

①学习基础知识



对于8259A芯片产生的中断，我们需要手动在中断返回前向8259A发送EOI消息。如果没有发送EOI消息，那么此后的中断便不会被响应。

一个发送EOI消息的示例代码如下， OCW2 在后面会介绍。

```
;发送OCW2字  
mov al, 0x20  
out 0x20, al  
out 0xa0, al
```

8259A的中断处理函数末尾必须加上面这段代码，否则中断不会被响应。

▲ICW

▲在任何时候 8259A 发送 **OCW** 字(Operation Command Words)来实现优先级、中断屏蔽字和 EOI 消息的动态改变。

▲8259A 芯片产生的中断，则需要在中断返回前发送 EOI 消息

②初始化 8259A 芯片：设置 ICW 字（未建立处理 8259A 中断的所有函数——屏蔽主片和从片的所有中断）

③编写中断处理函数：stdio.h+stdio.cpp+asm_time_interrupt_handler (跑马灯显示学号以及姓名首字母)

▲cpp 文件里面包含 c 语言代码也需要在函数前+extern “C”

```

1  extern "C" void c_time_interrupt_handler()
2  {
3      // 清空屏幕
4      for (int i = 0; i < 80; ++i)
5      {
6          stdio.print(0, i, ' ', 0x07);
7      }
8
9      const char text[] = "24325242txc";
10
11     int text_len = 0;
12     while(text[text_len] != '\0') text_len++;
13
14     //控制跑马灯偏移
15     int offset = times % 80;
16     times++;
17
18     stdio.moveCursor(offset); // 光标移动到偏移位置
19     for(int i = 0; i < text_len; i++)
20     {
21         if(offset + i < 80)
22         {
23             stdio.print(text[i]);
24         }
25     }
26 }

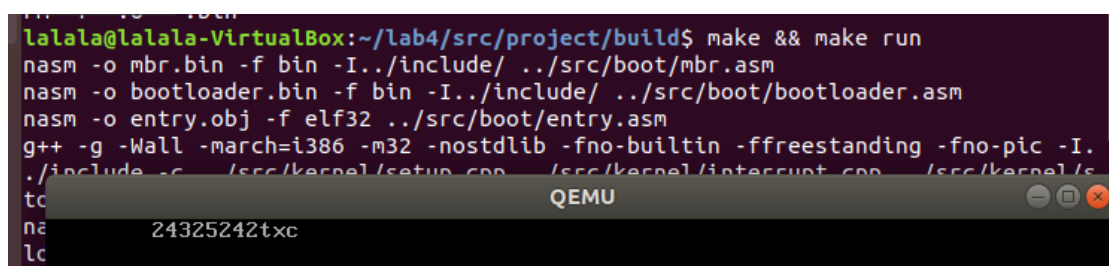
```

④设置时钟中断描述符

⑤封装开启和关闭时钟中断的函数：enableTimeInterrupt()+disableTimeInterrupt()

⑥在 setup_kernel 中定义 STDIO 的实例 stdio，最后初始化内核的组件，然后开启时钟中断和开中断

● 实验结果展示：



```

lalala@lalala-VirtualBox:~/lab4/src/project/build$ make && make run
nasm -o mbr.bin -f bin -I../include/ ../src/boot/mbr.asm
nasm -o bootloader.bin -f bin -I../include/ ../src/boot/bootloader.asm
nasm -o entry.obj -f elf32 ../src/boot/entry.asm
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I.
../include -I../src/kernel/setup.cpp -I../src/kernel/interrupt.cpp -I../src/kernel/s
to
QEMU
ne
24325242txc
ld

```

```
lalala@lalala-VirtualBox:~/lab4/src/project/build$ make && make run
nasm -o mbr.bin -f bin -I../include/ ../src/boot/mbr.asm
nasm -o bootloader.bin -f bin -I../include/ ../src/boot/bootloader.asm
nasm -o entry.obj -f elf32 ../src/boot/entry.asm
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I.
../include -I../src/kernel/setup.cpp -I../src/kernel/interrupt.cpp -I../src/kernel/s
tc
nasm -o 24325242t.xc
ld
```

Section 5 实验总结与心得体会

实验总结：本次实验围绕 x86 保护模式下的中断机制展开，学习了如何实现 C/C++/汇编的混合编程，实现了从“C 代码到内核可执行文件”的构建流程，并且完成了时钟中断的处理与跑马灯效果实现

实验心得：

1. 理解了程序如何运行，以及汇编代码与 C/C++ 在运用权限上的区别（汇编代码才可以初始化 IDT）
2. 理解了 os 是中断驱动的这个概念，时钟中断、键盘、硬盘等都依赖中断机制
3. 在本次实验中多次出先错误是因为未在 asm_utils.h 文件中声明 extern “C”，导致无法调用函数
4. 本次实验出现 image “raw” 问题（需要删除错误的 hd.img，再创建），以及未清理之前编译链接的文件导致 qemu 界面无任何输出

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学 2023 学年春季操作系统课程 - Gitee.com](#)